

JavaScript Asynchronous Programming

24.2 — Promises Deep Dive

Goal: এই document শেষ করার পর তুমি Promise কী, কীভাবে তৈরি হয়, কীভাবে settle হয়, `.then()`, `.catch()`, `.finally()` কীভাবে কাজ করে, Promise chaining কীভাবে value/error propagate করে এবং callback-based code কীভাবে Promise-based code-এ convert করতে হয়—এসব confidently বুঝতে পারবে।

1. Promise কী?

Promise হলো JavaScript-এর একটি object যা একটি asynchronous operation-এর eventual result বা failure represent করে।

সহজভাবে:

"এখন result নেই, কিন্তু ভবিষ্যতে result আসবে—অথবা error হবে।"

Example:

```
const promise = fetch("/api/users");  
  
console.log(promise);
```

এই মুহূর্তে network response পাওয়া না গেলেও `fetch()` একটা Promise return করে।

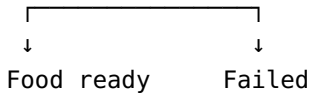
2. Real-Life Mental Model

ধরো তুমি restaurant-এ খাবার order করলে।

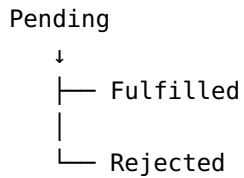
তখন খাবার সঙ্গে সঙ্গে যাতে নেই।

তুমি একটা expectation পেলো:

```
Order placed  
↓  
Waiting  
↓
```



JavaScript Promise-ও অনেকটা এমন।



3. Promise-এর তিনটি State

Promise-এর তিনটি primary state:

State	Meaning
pending	কাজ এখনো শেষ হয়নি
fulfilled	কাজ সফল হয়েছে
rejected	কাজ ব্যর্থ হয়েছে

4. Pending

Promise তৈরি হয়েছে কিন্তু এখনো result পাওয়া যায়নি।

```
const promise = new Promise((resolve, reject) => {  
  // কাজ চলছে  
});
```

এটা:

pending

state-এ থাকবে যতক্ষণ না `resolve()` অথবা `reject()` করা হয়।

5. Fulfilled

Operation সফল হলে:

```
resolve("Success");
```

Promise fulfilled হয়।

```
const promise = new Promise((resolve) => {  
  resolve("Success");  
});
```

এখন:

```
pending  
  ↓  
fulfilled
```

6. Rejected

Operation fail হলে:

```
reject("Something went wrong");
```

Promise rejected হয়।

```
const promise = new Promise((resolve, reject) => {  
  reject("Failed");  
});
```

Flow:

```
pending  
  ↓  
rejected
```

7. Promise একবার Settle হলে আর Change হয় না

Promise:

```
pending
  ↓
fulfilled
```

হতে পারে।

অথবা:

```
pending
  ↓
rejected
```

কিন্তু:

```
fulfilled → rejected
```

বা:

```
rejected → fulfilled
```

হতে পারে না।

Example:

```
const promise = new Promise((resolve, reject) => {
  resolve("First");

  reject("Second");
});

promise.then((value) => {
  console.log(value);
});
```

Output:

First

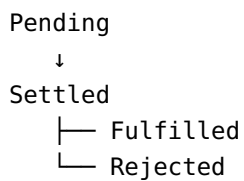
কারণ প্রথম settlement-টাই কার্যকর হয়েছে।

8. Settlement

`fulfilled` এবং `rejected` —দুটোকেই collectively বলা হয়:

Settled

অর্থাৎ:



একবার settled হলে Promise-এর state আর পরিবর্তন হয় না।

9. Creating a Promise

Basic syntax:

```
const promise = new Promise((resolve, reject) => {  
  // asynchronous operation  
  
  resolve(value);  
  // অথবা  
  reject(error);  
});
```

এখানে executor function:

```
(resolve, reject) => {  
  // ...  
}
```

automatically execute হয়।

10. resolve()

`resolve()` Promise-কে successful result দেয়।

```
const promise = new Promise((resolve) => {  
  resolve("Data received");  
});
```

তারপর:

```
promise.then((value) => {  
  console.log(value);  
});
```

Output:

```
Data received
```

11. reject()

`reject()` Promise-কে failure signal দেয়।

```
const promise = new Promise((resolve, reject) => {  
  reject(new Error("Something went wrong"));  
});
```

তারপর:

```
promise.catch((error) => {  
  console.error(error.message);  
});
```

Output:

Something went wrong

12. কেন `Error` `Object` ব্যবহার করা ভালো?

এটা:

```
reject("Failed");
```

কাজ করবে।

কিন্তু সাধারণত better:

```
reject(new Error("Failed"));
```

কারণ `Error` object-এর মধ্যে:

- `message`
- `name`
- `stack`

ইত্যাদি useful information থাকে।

13. `Promise Executor` কখন `Execute` হয়?

খুব important:

```
const promise = new Promise((resolve, reject) => {  
  console.log("Executor");  
});  
  
console.log("After");
```

Output:

```
Executor  
After
```

Executor function **Promise** তৈরি হওয়ার সময়ই **synchronously execute** হয়।

Promise asynchronous হওয়া মানেই executor automatically asynchronous নয়।

14. Promise নিজে Asynchronous নয়

এই distinction খুব important।

```
const promise = new Promise((resolve) => {  
  console.log("Inside");  
  
  resolve("Done");  
});  
  
console.log("Outside");
```

Output:

```
Inside  
Outside
```

Promise constructor-এর executor synchronous।

তবে `.then()`, `.catch()` ইত্যাদির callback asynchronousভাবে schedule হয়।

15. Promise.resolve()

Existing value থেকে fulfilled Promise তৈরি করা যায়:

```
const promise = Promise.resolve("Hello");
```

Equivalent concept:

```
const promise = new Promise((resolve) => {  
  resolve("Hello");  
});
```

তারপর:

```
promise.then((value) => {  
  console.log(value);  
});
```

16. Promise.reject()

Rejected Promise তৈরি:

```
const promise = Promise.reject(  
  new Error("Failed")  
);
```

Handle:

```
promise.catch((error) => {  
  console.error(error.message);  
});
```

17. .then()

Promise successful হলে `.then()` ব্যবহার করি।

```
fetchData()  
  .then((data) => {  
    console.log(data);  
  });
```

`.then()` callback fulfilled value পায়।

18. .catch()

Promise rejected হলে `.catch()` ব্যবহার করি।

```
fetchData()
  .then((data) => {
    console.log(data);
  })
  .catch((error) => {
    console.error(error);
  });
```

19. `.finally()`

Promise fulfilled বা rejected—দুই ক্ষেত্রেই সাধারণ cleanup কাজের জন্য `finally()` ব্যবহার করা যায়।

```
fetchData()
  .then((data) => {
    console.log(data);
  })
  .catch((error) => {
    console.error(error);
  })
  .finally(() => {
    console.log("Finished");
  });
```

Typical use cases:

- Loading indicator বন্ধ করা
- Resource cleanup
- UI state reset
- Logging

20. `.then()` সবসময় একটি নতুন **Promise Return** করে

এটা Promise chaining-এর foundation।

```
const promise1 = Promise.resolve(10);

const promise2 = promise1.then((value) => {
  return value * 2;
});
```

```
console.log(promise1);  
console.log(promise2);
```

দুটো আলাদা Promise।

```
promise1  
  ↓  
.then()  
  ↓  
promise2
```

21. `.then()` থেকে Value Return

```
Promise.resolve(10)  
  .then((value) => {  
    return value * 2;  
  })  
  .then((value) => {  
    console.log(value);  
  });
```

Output:

```
20
```

Flow:

```
10  
  ↓  
20
```

22. Promise Chaining

একাধিক asynchronous/successive operation ধারাবাহিকভাবে execute করতে Promise chain ব্যবহার করা হয়।

```

getUser()
  .then((user) => {
    return getPosts(user.id);
  })
  .then((posts) => {
    return getComments(posts[0].id);
  })
  .then((comments) => {
    console.log(comments);
  });

```

Flow:

```

getUser()
  ↓
user
  ↓
getPosts()
  ↓
posts
  ↓
getComments()
  ↓
comments

```

23. কেন `return` Important?

এই code:

```

getUser()
  .then((user) => {
    return getPosts(user.id);
  })
  .then((posts) => {
    console.log(posts);
  });

```

এখানে প্রথম `.then()` থেকে Promise return করা হয়েছে।

তাই পরের `.then()` সেই Promise-এর result পাবে।

24. `return` না করলে কী হয়?

Wrong:

```
getUser()
  .then((user) => {
    getPosts(user.id);
  })
  .then((posts) => {
    console.log(posts);
  });
```

এখানে `getPosts()` `return` করা হয়নি।

তাই পরের `.then()` সাধারণত `undefined` পাবে।

Correct:

```
getUser()
  .then((user) => {
    return getPosts(user.id);
  })
  .then((posts) => {
    console.log(posts);
  });
```

25. Returning Normal Value

```
Promise.resolve(5)
  .then((value) => {
    return value + 10;
  })
  .then((value) => {
    console.log(value);
  });
```

Output:

15

Conceptually JavaScript এটাকে পরের chain-এর জন্য fulfilled value হিসেবে ব্যবহার করে।

26. Returning a Promise

```
Promise.resolve(5)
  .then((value) => {
    return Promise.resolve(value * 2);
  })
  .then((value) => {
    console.log(value);
  });
```

Output:

10

এখানে পরের `.then()` inner Promise-এর result-এর জন্য অপেক্ষা করে।

এটাকে Promise assimilation/flattening-এর simplified mental model দিয়ে বোঝা যায়।

27. Promise Chain Flattening

ধরো:

```
const promise = Promise.resolve(10)
  .then(() => {
    return Promise.resolve(20);
  });
```

তাহলে:

```
promise.then((value) => {
  console.log(value);
});
```

Output:

20

তুমি সাধারণত nested Promise structure নিয়ে কাজ করতে হয় না।

28. `.then()` এর দুই Callback

`.then()` এই form-এও লেখা যায়:

```
promise.then(  
  (value) => {  
    // fulfilled  
  },  
  (error) => {  
    // rejected  
  }  
);
```

Example:

```
Promise.resolve("Success").then(  
  (value) => {  
    console.log(value);  
  },  
  (error) => {  
    console.error(error);  
  }  
);
```

তবে সাধারণ code-এ error handling-এর জন্য:

```
promise  
  .then(...)  
  .catch(...);
```

অনেক বেশি readable।

29. `.catch()` আসলে কী?

Conceptually:

```
promise.catch(handleError);
```

অনেকটা:

```
promise.then(undefined, handleError);
```

এর মতো।

কিন্তু chain-এর error propagation-এর কারণে `.catch()` অনেক বেশি useful এবং readable।

30. Error Propagation

Promise chain-এর বড় সুবিধা:

Error chain-এর মধ্যে নিচের দিকে propagate করতে পারে।

Example:

```
Promise.resolve()
  .then(() => {
    throw new Error("Something failed");
  })
  .then(() => {
    console.log("This will not run");
  })
  .catch((error) => {
    console.error(error.message);
  });
```

Output:

```
Something failed
```

31. `throw` করলে কী হয়?

`.then()` callback-এর মধ্যে:

```
throw new Error("Failed");
```

করলে returned Promise rejected হয়ে যায়।

Concept:

```
.then()  
  ↓  
throw Error  
  ↓  
Rejected Promise  
  ↓  
.catch()
```

32. `.catch()` দিয়ে **Recovery**

`.catch()` শুধু error print করার জন্য নয়।

এটা error recover করতেও পারে।

```
Promise.reject(new Error("Failed"))  
  .catch(() => {  
    return "Fallback value";  
  })  
  .then((value) => {  
    console.log(value);  
  });
```

Output:

```
Fallback value
```

এখানে:

```
Rejected
  ↓
catch()
  ↓
Fulfilled with "Fallback value"
```

33. `.catch()` থেকেও **Error Throw** করা যায়

```
Promise.reject(new Error("Original error"))
  .catch((error) => {
    console.log(error.message);

    throw new Error("New error");
  })
  .catch((error) => {
    console.log(error.message);
  });
```

Output:

```
Original error
New error
```

34. Error Propagation Diagram

```
Promise
  ↓
.then()
  ↓
.then()
  ↓
Error
  ↓
.catch()
  ↓
Recovery
```

↓
.then()

Error হলে successful `.then()` callbacks skip হতে পারে যতক্ষণ না appropriate rejection handler পাওয়া যায়।

35. Error Handler-এর পর Chain আবার Continue করতে পারে

```
Promise.reject(new Error("Failed"))
  .catch(() => {
    console.log("Recovered");

    return "Fallback";
  })
  .then((value) => {
    console.log(value);
  });
```

Output:

Recovered
Fallback

কারণ `.catch()` successful value return করেছে।

36. `.finally()` কীভাবে কাজ করে?

```
Promise.resolve("Success")
  .finally(() => {
    console.log("Cleanup");
  })
  .then((value) => {
    console.log(value);
  });
```

Output:

```
Cleanup  
Success
```

37. `finally()` সাধারণত **Result Change** করে না

```
Promise.resolve("Hello")  
  .finally(() => {  
    console.log("Cleanup");  
  })  
  .then((value) => {  
    console.log(value);  
  });
```

Output:

```
Cleanup  
Hello
```

`finally()` callback-এর return value সাধারণত আগের result replace করে না।

38. `finally()` **Error** হলেও **Preserve** হতে পারে

```
Promise.reject(new Error("Failed"))  
  .finally(() => {  
    console.log("Cleanup");  
  })  
  .catch((error) => {  
    console.log(error.message);  
  });
```

Output:

```
Cleanup  
Failed
```

39. `finally()` -এর Important Exception

যদি `finally()` নিজেই throw করে:

```
Promise.resolve("Success")
  .finally(() => {
    throw new Error("Cleanup failed");
  })
  .then((value) => {
    console.log(value);
  })
  .catch((error) => {
    console.log(error.message);
  });
```

Output:

```
Cleanup failed
```

অর্থাৎ `finally()` chain-এর outcome affect করতে পারে যদি সেখানে error throw হয় বা rejected Promise return হয়।

40. Promise vs Callback

Callback:

```
getUser((error, user) => {
  if (error) {
    handleError(error);
    return;
  }

  getPosts(user.id, (error, posts) => {
    if (error) {
      handleError(error);
      return;
    }

    console.log(posts);
  });
});
```

Promise:

```
getUser()
  .then((user) => getPosts(user.id))
  .then((posts) => {
    console.log(posts);
  })
  .catch(handleError);
```

Promise version-এর major advantage:

- chaining
- centralized error handling
- composition
- better readability
- easier async control flow

41. Promise Constructor কখন ব্যবহার করবে?

সবসময় `new Promise()` লিখতে হবে—এমন নয়।

Example:

```
function getData() {
  return new Promise((resolve, reject) => {
    // custom async operation
  });
}
```

যখন callback/event-based API-কে Promise-এ convert করতে হবে, তখন constructor useful।

কিন্তু যদি already Promise থাকে:

```
function getData() {
  return fetch("/api/data");
}
```

তখন আবার:

```
return new Promise(...)
```

দিয়ে wrap করার প্রয়োজন নেই।

42. Promise Constructor Anti-Pattern

Unnecessary wrapping:

```
function getData() {  
  return new Promise((resolve, reject) => {  
    fetch("/api/data")  
      .then((response) => {  
        resolve(response);  
      })  
      .catch((error) => {  
        reject(error);  
      });  
  });  
}
```

এটা unnecessary।

Better:

```
function getData() {  
  return fetch("/api/data");  
}
```

অথবা যদি transformation দরকার হয়:

```
function getData() {  
  return fetch("/api/data")  
    .then((response) => response.json());  
}
```

43. Promise-এর Asynchronous Callback

এই code:

```
console.log("A");

Promise.resolve().then(() => {
  console.log("B");
});

console.log("C");
```

Output:

```
A
C
B
```

কারণ `.then()` callback synchronousভাবে call হয় না।

এটা পরে **Microtask Queue** topic-এ deepভাবে আসবে।

44. Promise Chain Example

```
function getUser() {
  return Promise.resolve({
    id: 1,
    name: "Shariar",
  });
}

function getPosts(userId) {
  return Promise.resolve([
    {
      id: 101,
      userId,
      title: "JavaScript",
    },
  ]);
}

getUser()
  .then((user) => {
    console.log("User:", user);

    return getPosts(user.id);
```

```

    })
    .then((posts) => {
      console.log("Posts:", posts);
    })
    .catch((error) => {
      console.error("Error:", error);
    })
    .finally(() => {
      console.log("Finished");
    });

```

Flow:

```

getUser()
  ↓
user
  ↓
getPosts(user.id)
  ↓
posts
  ↓
catch if error
  ↓
finally

```

45. Promise Chain-এর Golden Rule

সবচেয়ে important rule:

যদি পরের `.then()` -কে আগের **asynchronous operation**-এর **result**-এর জন্য অপেক্ষা করাতে চাও, **Promise** **return** করো।

Example:

```

.then(() => {
  return fetchData();
})

```

অথবা concise:

```
.then(() => fetchData())
```

46. Multiple `.then()` on Same Promise

```
const promise = Promise.resolve("Hello");

promise.then((value) => {
  console.log("A:", value);
});

promise.then((value) => {
  console.log("B:", value);
});
```

দুটো independent reaction ।

এগুলোকে chain ভাবে না ।

Chain:

```
promise
  .then(...)
  .then(...);
```

Multiple consumers:

```
promise.then(...);
promise.then(...);
```

দুটো আলাদা concept ।

47. Promise Reuse

একটা Promise multiple জায়গা থেকে consume করা যায় ।

```
const dataPromise = fetch("/api/data");
```

```
const firstConsumer = dataPromise.then(...);
const secondConsumer = dataPromise.then(...);
```

এতে একই underlying Promise object-এর result multiple consumers পেতে পারে।

48. Promise State Inspect করা যায়?

JavaScript-এর standard API দিয়ে সরাসরি:

```
promise.state
```

এরকম কিছু নেই।

তুমি সাধারণত Promise-এর ওপর:

```
.then()
.catch()
.finally()
```

দিয়ে behavior observe করো।

49. Promise vs Promise Result

এই distinction খুব important।

```
const promise = Promise.resolve("Hello");
```

`promise` হলো:

```
Promise object
```

কিন্তু:

```
promise.then((value) => {
  console.log(value);
});
```

এখানে `value` হলো:

```
"Hello"
```

অর্থাৎ:

```
Promise  
  ↓  
eventual value
```

Promise নিজে value নয়।

50. Common Mistake — Promise-কে Value ভাবা

Wrong mental model:

```
const user = getUser();  
  
console.log(user.name);
```

যদি `getUser()` Promise return করে, তাহলে `user` সরাসরি user object নয়।

বরং:

```
const userPromise = getUser();
```

তারপর:

```
userPromise.then((user) => {  
  console.log(user.name);  
});
```

অথবা পরে:

```
const user = await getUser();
```

51. Promise Resolution Procedure — Advanced Mental Model

এটা একটু advanced, কিন্তু জানা important ।

`resolve(value)` মানেই সবসময় immediately simple fulfillment নয় ।

যদি:

```
resolve(10);
```

তাহলে Promise eventually value `10` দিয়ে fulfill হবে ।

কিন্তু:

```
resolve(anotherPromise);
```

হলে outer Promise অন্য Promise-এর state/result follow করতে পারে ।

Concept:

```
Outer Promise
  ↓
resolve(inner Promise)
  ↓
wait/follow inner Promise
  ↓
final outcome
```

এটাই Promise resolution-এর deeper behavior-এর অংশ ।

52. Thenable

JavaScript Promise শুধু actual Promise object-এর সাথে কাজ করে না ।

Promise-like object-কে **thenable** বলা হয় ।

Example:

```
const thenable = {
  then(resolve) {
    resolve("Hello");
  },
};
```

Promise resolution machinery thenable-এর `then` method recognize করতে পারে।

এই কারণেই Promise resolution system একটু sophisticated।

53. Promise Assimilation

Example:

```
Promise.resolve({
  then(resolve) {
    resolve("Hello");
  },
});
```

এখানে Promise thenable-এর behavior follow করতে পারে।

এই advanced concept future Promise-heavy libraries এবং APIs বুঝতে useful।

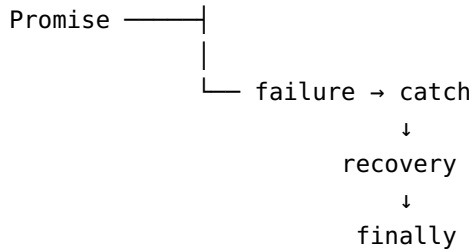
54. Error Handling Strategy

Basic Promise chain:

```
doSomething()
  .then(stepOne)
  .then(stepTwo)
  .then(stepThree)
  .catch(handleError)
  .finally(cleanup);
```

Mental model:

```
└─ success → next then
|
```



55. `catch()` Placement

এটা:

```
doA()  
  .then(doB)  
  .then(doC)  
  .catch(handleError);
```

পুরো chain-এর error handle করতে পারে।

কিন্তু কখনো specific error locally handle করতে হতে পারে:

```
doA()  
  .then(doB)  
  .catch(handleBError)  
  .then(doC)  
  .catch(handleGeneralError);
```

কোন error কোথায় recover হবে—এটা architecture-এর ব্যাপার।

56. Promise Chain Debugging

যদি chain কাজ না করে:

Step 1

প্রতিটা stage-এর value log করো।

```
getUser()  
  .then((user) => {
```

```
    console.log("User:", user);
    return getPosts(user.id);
  })
  .then((posts) => {
    console.log("Posts:", posts);
    return posts;
  })
  .catch((error) => {
    console.error(error);
  });
```

Step 2

Check করে:

Promise return করছি কি না?

Step 3

Check করে:

Error কোথায় reject হচ্ছে?

57. Practical API Pattern

```
function getUser() {
  return fetch("/api/user")
    .then((response) => {
      if (!response.ok) {
        throw new Error("Failed to fetch user");
      }

      return response.json();
    });
}
```

তারপর:

```
getUser()
  .then((user) => {
    console.log(user);
  })
  .catch((error) => {
```

```
    console.error(error);  
  });
```

Important:

`fetch()` HTTP `4xx/5xx` status-এর জন্য নিজে automatically reject করে না।

তাই:

```
if (!response.ok) {  
  throw new Error(...);  
}
```

pattern important।

58. Promise Chain vs Nested Promise

Bad:

```
getUser().then((user) => {  
  getPosts(user.id).then((posts) => {  
    console.log(posts);  
  });  
});
```

Better:

```
getUser()  
  .then((user) => {  
    return getPosts(user.id);  
  })  
  .then((posts) => {  
    console.log(posts);  
  });
```

কারণ nested Promise chain callback nesting-এর মতো complexity তৈরি করতে পারে।

59. Promise Best Practices

1. Promise return করো

```
.then(() => fetchData())
```

2. Error handle করো

```
.catch(handleError)
```

3. Already-Promise-কে unnecessary wrap করো না

```
return fetch(url);
```

4. `new Promise()` শুধু প্রয়োজন হলে ব্যবহার করো

5. Meaningful errors throw করো

```
throw new Error("User not found");
```

6. Chain readable রাখো

অনেক nested `.then()` avoid করো।

7. Independent Promise-এর জন্য পরে **combinators** ব্যবহার করো

```
Promise.all(...)
```

60. Practice — Level 1

Question 1

Promise-এর তিনটি state কী?

Question 2

`fulfilled` এবং `settled` কি একই জিনিস?

Explain করো।

Question 3

Promise কি একবার `fulfilled` হওয়ার পর `rejected` হতে পারে?

Question 4

`resolve()` এবং `reject()` কী করে?

Question 5

`.then()`, `.catch()`, `.finally()`—তিনটির purpose কী?

61. Practice — Level 2

Question 6

একটা Promise বানাও যেটা 2 seconds পরে:

"Data loaded"

return করবে।

Question 7

একটা Promise বানাও যেটা 1 second পরে reject করবে:

Error("Something went wrong")

দিয়ে।

Question 8

নিচের callback code-কে Promise version-এ convert করো:

```
function getData(callback) {  
  setTimeout(() => {  
    callback(null, "Hello");  
  }, 1000);  
}
```

62. Practice — Promise Chaining

Question 9

এই flow implement করো:

```
getUser()  
  ↓  
getPosts(user.id)  
  ↓  
getComments(posts[0].id)
```

Promise chaining ব্যবহার করে।

Question 10

নিচের code-এ bug কোথায়?

```
getUser()  
  .then((user) => {  
    getPosts(user.id);  
  })  
  .then((posts) => {  
    console.log(posts);  
  });
```

কীভাবে ঠিক করবে?

63. Practice — Error Handling

Question 11

নিচের code-এ error কোথায় যাবে?

```
Promise.resolve()  
  .then(() => {  
    throw new Error("A");  
  })  
  .then(() => {  
    console.log("B");  
  })  
  .then(() => {  
    console.log("C");  
  })  
  .catch((error) => {  
    console.log(error.message);  
  });
```

Output কী?

Question 12

এখানে final output কী?

```
Promise.reject(new Error("Failed"))  
  .catch(() => {  
    return "Recovered";  
  })  
  .then((value) => {  
    console.log(value);  
  });
```

64. Practice — finally()

Question 13

Output কী?

```
Promise.resolve("Success")
  .finally(() => {
    console.log("Cleanup");
  })
  .then((value) => {
    console.log(value);
  });
```

Question 14

Output কী?

```
Promise.reject(new Error("Failed"))
  .finally(() => {
    console.log("Cleanup");
  })
  .catch((error) => {
    console.log(error.message);
  });
```

65. Practice — Tricky Output

Question 15

Output predict করো:

```
console.log("A");

Promise.resolve("B")
  .then((value) => {
    console.log(value);
    return "C";
  })
  .then((value) => {
    console.log(value);
  });

console.log("D");
```

Question 16

Output की?

```
const promise = new Promise((resolve, reject) => {
  console.log("Executor");

  resolve("First");
  resolve("Second");
  reject(new Error("Error"));
});

promise.then((value) => {
  console.log(value);
});

console.log("Done");
```

66. Practice — Advanced

Question 17

Explain the difference:

```
const a = getUser();

a.then((user) => {
  console.log(user);
});
```

and:

```
const user = await getUser();
```

Question 18

Why does this code behave differently?

```
getUser()
  .then((user) => {
    return getPosts(user.id);
  })
  .then((posts) => {
    console.log(posts);
  });
```

vs:

```
getUser()
  .then((user) => {
    getPosts(user.id);
  })
  .then((posts) => {
    console.log(posts);
  });
```

67. Challenge — Build a Promise-based API

একটা function তৈরি করো:

```
getUser(id)
```

Requirements:

- 1 second delay
- `id === 1` হলে user return
- অন্য ID হলে reject
- `Error` object ব্যবহার করতে হবে

Expected:

```
getUser(1)
  .then((user) => {
    console.log(user);
  })
  .catch((error) => {
    console.error(error.message);
  });
```

68. Challenge — Promise Chain

তিনটা fake API তৈরি করো:

```
getUser()  
getPosts(userId)  
getComments(postId)
```

তারপর:

```
User  
↓  
Posts  
↓  
Comments
```

Promise chaining দিয়ে implement করো।

সব error একটা `.catch()` দিয়ে handle করার চেষ্টা করো।

69. Challenge — Recovery

একটা Promise reject করো।

তারপর `.catch()` দিয়ে:

```
Fallback Data
```

return করো।

তারপর পরের `.then()` থেকে সেই fallback data print করো।

Flow:

```
Rejected  
↓  
catch()  
↓
```

Fallback

↓

then()

70. Interview Questions

Basic

1. Promise কী?
2. Promise-এর states কী কী?
3. Pending এবং settled-এর difference কী?
4. `resolve()` কী?
5. `reject()` কী?
6. Promise একবার settle হওয়ার পর state change হয় না কেন?
7. `.then()` কী করে?
8. `.catch()` কী করে?
9. `.finally()` কী করে?

Intermediate

1. `.then()` কি Promise return করে?
2. Promise chaining কী?
3. `.then()` থেকে value return করলে কী হয়?
4. `.then()` থেকে Promise return করলে কী হয়?
5. Promise chain-এ error কীভাবে propagate করে?
6. `.catch()` কি error recover করতে পারে?
7. `finally()` কি previous value replace করে?
8. Promise constructor executor কি synchronous?

Advanced

1. `new Promise()` কখন ব্যবহার করা উচিত?
2. Promise constructor anti-pattern কী?
3. Promise assimilation কী?
4. Thenable কী?
5. Multiple `.then()` এবং chaining-এর difference কী?
6. Promise নিজে asynchronous নাকি Promise callbacks asynchronous?
7. `fetch()` HTTP error-এর জন্য কেন automatically reject করে না?
8. Promise এবং async/await-এর relationship কী?

71. Quick Revision

Promise

→ Future result/failure represent করে

Pending

→ এখনো settled হয়নি

Fulfilled

→ সফলভাবে completed

Rejected

→ failed

Settled

→ fulfilled অথবা rejected

resolve()

→ success

reject()

→ failure

then()

→ fulfilled result handle

catch()

→ rejection/error handle

finally()

→ cleanup

.then()

→ নতুন Promise return করে

return value

→ পরের then সেই value পায়

return Promise

→ পরের then সেই Promise-এর result-এর জন্য wait করে

throw

→ current chain-এর returned Promise reject করতে পারে

catch()

→ error recover করতে পারে

Promise chaining

→ sequential async operations cleanভাবে compose করা

Promise.resolve()

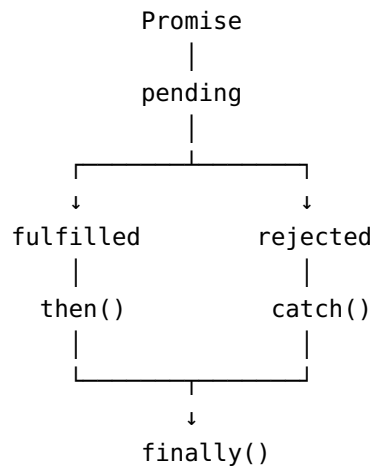
→ fulfilled Promise তৈরি

Promise.reject()

→ rejected Promise তৈরি

72. The Most Important Mental Model

Promise-কে এভাবে ভাবো:



অথবা chaining:

```
Promise A
  ↓
  .then()
  ↓
Promise B
  ↓
  .then()
  ↓
Promise C
  ↓
  .catch()
```

↓
finally()

সবচেয়ে important rule:

Promise chain-এ পরের **step**-কে আগের **async operation**-এর **result** দিতে চাইলে **Promise return** করতে হবে।

✓ Mastery Checklist

- ☐ Promise কী বুঝি
- ☐ Promise-এর 3 states বুঝি
- ☐ Pending/settled বুঝি
- ☐ Fulfilled/rejected বুঝি
- ☐ `resolve()` বুঝি
- ☐ `reject()` বুঝি
- ☐ Promise settlement বুঝি
- ☐ Promise executor synchronous হওয়া বুঝি
- ☐ `Promise.resolve()` বুঝি
- ☐ `Promise.reject()` বুঝি
- ☐ `.then()` বুঝি
- ☐ `.catch()` বুঝি
- ☐ `.finally()` বুঝি
- ☐ `.then()` নতুন Promise return করে বুঝি
- ☐ Value return vs Promise return বুঝি
- ☐ Promise chaining বুঝি
- ☐ Error propagation বুঝি
- ☐ Error recovery বুঝি
- ☐ `finally()` behavior বুঝি
- ☐ Promise constructor কখন দরকার বুঝি
- ☐ Promise constructor anti-pattern বুঝি
- ☐ Thenable-এর basic ধারণা আছে
- ☐ Promise assimilation-এর basic ধারণা আছে
- ☐ Callback → Promise migration বুঝি
- ☐ Promise output prediction করতে পারি

Next Document

24.3 – Async/Await & Concurrency

এখানে আমরা Promise-এর ওপর দাঁড়িয়ে শিখব:

```
async / await
  ↓
try / catch
  ↓
Promise.all()
Promise.allSettled()
Promise.race()
Promise.any()
  ↓
Sequential vs Parallel
  ↓
Async/Await in loops
  ↓
Concurrency
  ↓
Real-world API patterns
```

এই document-এ `forEach(async...)` কেন ভাল/ব্যক্তিপূর্ণ, `map + Promise.all()`, sequential বনাম parallel execution এবং চারটা Promise combinator-এর practical decision-making খুব গভীরভাবে করব।